How the Agents Got Their Present Moment

One small step for operational semantics, one giant step for agentkind

Lucius Gregory Meredith

Jan 15, 2026

1 Introduction and motivation

It's been 21 years since the publication of the first paper on the `rho calculus`. In that time a lot has happened to this line of research – not the least of which is that a lean, yet production-ready programming language and execution environment based on the calculus has been developed and commercially deployed in several projects. This effort has driven the investigation of many variants of the calculus. And, with the resurgence of interest in AI and agentic computation it was only a matter of time before a variant particularly well suited for agentic AI would be discovered. This paper reports on that variant.

The key idea is to introduce a slight modification to the core rewrite rule of the calculus. This modification works by allowing a `rho calculus` agent to balance between the present and the future. An agent's present is defined by all of the interactions it might have, either internally or with its environment, in a single step from its current state. These interactions are transactional, meaning they effect a potentially irreversible state change and prune options for interaction. In contrast, an agent may peer into its future. This precognition does not prune potential interactions, but rather keeps the branching structure around and available for the agent's introspection and decision-making.

The variant was discovered for a completely different purpose related to the introduction of a sophisticated discipline for sending typed messages on channels. In a nutshell, the idea underlying the typed message discipline is to expand the `rho calculus`' builtin feature allowing agents to send the code of agents as messages. Specifically, to allow agents to send code from any model of computation, not just the `rho calculus`. More on this and the deeper rationale for this feature later. The crucial point is that if code from another model of computation – that is, other than the `rho calculus` – is being communicated, how does the code ever get run?

In the context of the `rho calculus` there are two natural alternatives. Of the two, one introduces into the act of communicating the code an ability to simulate the behavior of the code by n steps into the future. That is, both the code and some number of steps to run it for are part of the message sent. The receiver picks up the set of states (i.e. terms, i.e. code) the code can reach in at most n steps. But this feature could be retrofitted to the original calculus, i.e. one that didn't add the complexity of defining theories of computation and sending terms in them, but instead only allows agents to send and receive the code of agents. In this variant of the calculus, when n is 0 we recover the original `rho calculus`. When n is greater than 0 agents get a kind of precognitive ability to peer into the future of the code of an agent.

Now, such a feature, closely related to delimited continuations, could be costly, especially because peering into the future in a language that is not confluent comes with the inherent need to deal with copying lots of state. This can be remedied by introducing a trie-based storage mechanism for speculative exploration of state space. Trie-based storage provides an optimal compression technique for just these kinds of situations. More on this later, as well.

If you, dear reader, share the author's sensibilities, there's been far too much discussion and not enough math; so, let's render it into symbols. The original core engine of computation in the `rho calculus` is known as the `comm` (short for communication) rule. It looks like this

$$\text{for}(y \leftarrow x)P \mid x!(Q) \rightarrow P\{@Q/y\} \quad (\texttt{COMM}) \tag{1}$$

In English: an agent that will run P after receiving a message y on the channel x running concurrently with an agent that is sending the code of Q on channel x reduces in a single step to an instance of P in which every mention of y has been replaced by the code of Q, written $@Q$.

The variant under consideration modifies the rule as follows

$$Q \xrightarrow{n} \{Q_1, \ldots, Q_m\} \implies \text{for}(y \leftarrow x)P \mid x!(Q)[n] \rightarrow P\{@\{Q_1, \ldots, Q_m\}/y\} \quad \texttt{(LOOKAHEAD)} \quad (2)$$

In English: if Q reduces in at most n steps to any of $\{Q_1, \ldots, Q_m\}$, then an agent that will run P after receiving a message y on the channel x running concurrently with an agent that is sending the n-step future of the code of Q (i.e. the states reachable in at most n steps from Q) on channel x reduces in a single step to an instance of P in which every mention of y has been replaced by the code of the set $\{Q_1, \ldots, Q_m\}$, written $@\{Q_1, \ldots, Q_m\}$.

The astute reader will have spotted a potential circularity. In the original rule everything works because the definition of the reduction relation, $\rightarrow$, depends on nothing (apart from the mechanics of substitution and channel equality). In this new rule the definition of the rule seems to depend on the rule! We need to consult how Q evolves, but surely that will appeal to the consequent of the rule. As with many procedures, processes, and algorithms in life 0 comes to our rescue, for $Q \xrightarrow{0} Q$! So, as long as we have a way to extract Q from $\{Q\}$ the recursion in this formulation grounds out in the original rule.

That's the idea, really, and, as such, it would be grand to save both the reader and the author some time and end the paper here. Certainly, this account can be considered the TL;DR for the impatient. As it happens, however, there are several not so obvious consequences that follow from this variant that have considerable import for agentic computation and for an account of artificial intelligence, more generally. The rest of the paper is mostly an account of those consequences.

2 Related work

As mentioned above this feature is closely related to the work of Dybvig, Peyton-Jones, and Sabry on a monadic account of delimited continuations. Of course, their work is firmly situated in the context of confluent and sequential rather than concurrent computation (the λ-lambda calculus), and relies heavily on denotational and categorical techniques. The `rho calculus`, by contrast, is aimed at capturing computational phenomena more closely aligned with the real world, where people can race to get the last seat on an airline flight and there is a clear winner and a clear loser, where the competition for resources drives evolution in species, and the competition amongst molecules gives rise to a variety of chemical and biological phenomena. In other words, computational phenomena that do not enjoy confluence.

Likewise, operational semantics and rewriting, as opposed to denotational semantics, is emerging as a clear winner not only in implementation of programming languages, but in fields as far ranging as quantum mechanics (see the ZX-Calculus).

It should be mentioned that our work on the semantics of the agentic AI language `MeTTa` has influenced this work. It was a close analysis of how to compile `MeTTa` into `rholang` efficiently that led to these discoveries. With this small change, `rholang` is able to subsume all the features of `MeTTa` and provide significantly more. More on this later, too.

3 Outline of the content

We begin with a manifest of the technical apparatus, moving to the basic modifications of the calculus and what that does to bisimulation and typing. Then we look at how this impacts agentic reasoning in the `rho calculus` with the `lookahead` rule, which we hereby nickname the *spice calculus*, in reference to Dune and the application of the spice to the use of navigating hyperspace. For brevity we may also refer to the `lookahead` rule as the *spice rule*. Next we place the semantics in the context of `MeTTaIL`, a variant of `rholang` that affords programmers with the ability to reason about whole models of computation.

4 Technical presentation

4.1 A review of the rho calculus

4.1.1 The Mobile Process Calculi

Amongst all the models of computation – and there are infinitely many and hundreds and hundreds under active investigation by various research communities – there is one family of models that enjoys all four of the following properties: compositionality, concurrency, complexity, and completeness. The mobile process calculi.

Broadly speaking they are so named because

- They focus on processes rather than functions; that is to say while they can describe input/output relationships, they are not limited to this view of computational phenomena. For example, they can describe protocols, such as the sliding window protocol, which underlies TCP and all of the Internet. Indeed, to see how this might be a shift from a functional input/output view, ask yourself, what does the Internet compute? Does it have an output? Processes cover these and other cases that don't fit nicely into these earlier and frankly more naive models of computations. Instead processes formalize the idea that the network is the computer.

- What makes them 'mobile' processes is that this network is not fixed. It changes as processes evolve. This is not unlike human processes where two people randomly bump into one another at a conference and exchange email addresses or phone numbers or connect on WhatsApp. The communication topology, specifically the links in the who can communicate with whom network has changed. Nor is it unlike biochemical or biological processes where molecules randomly bump into one another and "hook up" or bond becoming a new molecule capable of interacting with other molecules that either would not have interacted with at all or not in the same way before the bond.

Among the mobile process calculi Robin Milner's π-calculus is perhaps most well known and certainly paradigmatic. One way to understand the π-calculus is that it builds up a notion of process from very basic message-passing activities.

- `0` – The simplest agent is the one that does nothing at all.

- `x!( m )` – The next simplest is one that simply fires off a message and that's it. Sending a message requires the message (m in the expression here) being sent and a channel (sometimes also called a port or a name) on which the message is sent (x in the expression here).

- `for( m <- x )P` – The next simplest after that is receiving a message. In the π-calculus a process listening for a message listens on a channel (x in the expression) and upon receipt continues as a new process (P in the expression) that has at its disposal the information contained in the message (m in the expression).

- `(new x)P` – To this core of message-passing primitives the π-calculus adds the ability to make fresh (alternatively *private*) channels.

- `!P` – The ability for a process to repeat a behavior, more specifically, to make copies of itself.

- `P|Q` – A process may comprise a collection of processes that are running concurrently.

The astute reader will have noticed that we haven't yet said what a channel, or indeed what a message is. The answers provided by the calculus are quite compelling in their own way. A message is nothing more nor less than a channel! In the π-calculus all computation arises in terms of how the network reconfigures itself. This often takes some getting used to, but this effort is required to make the idea that the network is the computer both mathematically precise and compositional.

Equally interesting is that the π-calculus does not say concretely what a channel is. Rather, it demands properties of any collection of gadgets that might serve as channels.

- First, there has to be an effective test to see when two channels are really the same channel. For example, if channels are email addresses Alice and Bob need to be able to compare email two email addresses to see if they are really the same address.

- Another property is that there needs to be at least countably infinitely many of them. For example there are countably infinitely many email addresses. In finite time we will only ever explore finitely many of them, but in principle there are as many email addresses as there are counting numbers.
- Finally, the π-calculus posits that names have no internal structure.

Taken together these three properties rob the π-calculus of the possibility of being concrete, or realizable on a physical substrate. It's a logical model that can have no physical counterpart, and so cannot be part of a computer.

The reason is simple. Without internal structure to exploit the only equality algorithm available is an infinite table, the rows and columns of which are the set of names, respectively. This table doesn't fit inside anything classically physical, though possibly something quantum mechanical.

To make a realization of the π-calculus on silicon-based hardware, people break this third requirement and give names internal structure so that they can implement the effective equality check in terms of channels' internal structure. For example, they make channels out of character strings. There are, in principle, infinitely many finite length character strings over an alphabet of characters (such as ASCII or Unicode), and there's an obvious effective procedure for testing whether two strings are the same. However, that procedure relies on a channel comprising a sequence of characters. Similarly, the set of channels could be the set of counting numbers. $\{0, 1, 2, 3, \dots\}$. There is an obvious effective procedure for testing if two numbers are equal.

The problem with these approaches is that under the guise of providing a theory of names we have unwittingly smuggled in a theory of computation. In the case of counting numbers we know that this is how Gödel smuggled in the theory of arithmetic into logic which formed the basis of his construction of a true sentence that was unprovable. Intriguingly, his construction involved a self-referential loop where the sentence refers to itself.

4.2 Reflection and Concurrency

Rather than seeing this strange loop as a bug, what distinguishes the `rho calculus` from the π-calculus is that it embraces reflection as a feature. In fact this is where it gets its name the *r*eflective *h*igher *o*rder calculus, or `rho calculus` for short. Pleasantly, the Greek letter for `rho`, ρ, comes immediately after π in the Greek alphabet.

The `rho calculus` abstracts Gödel's construction and introduces two operators, @ and $*$, that are dual to each other. The first turns a process into a channel, and the second turns a channel back into a process. Every channel arises as the code of some process, written $@P$ (one would not be far off thinking of this as the Gödel number of the process, but that is only one implementation amongst many many). Hence we don't need to smuggle in some other theory of channels into the theory of processes. We have everything we need already.

Additionally, we can now send processes as messages! What we send over the channel is really the code of the process, but the receiving process can always turn the code back into a process on demand. Further, with this new apparatus we can implement copying. And it turns out we can implement the generation of fresh names. Thus, the `rho calculus` not only fixes a serious problem with the π-calculus — one which either prevents it from being physically realizable or failing to provide a foundational account of concurrent computing — but also streamlines the calculus into something much simpler and more tractable.

Recapping, the `rho calculus` comprises the following syntactic categories of processes and names.

```
P,Q ::= 0 | x!( P ) | for( y <- x )P | P|Q | *x          x,y ::= @P
```

These, plus a small number of equations and rewrite rules, delivers a Turing complete model of computation that maps efficiently onto many different physical substrates. The equations are just what is necessary to impose alpha-equivalence on the binding operation in for-comprehensions, and make $(P, |, 0)$ into a commutative monoid. There is one base rewrite rule and two context-dependent rewrite rules.

$$\texttt{for(y <- x)P | x!(Q)} \to \texttt{P\{ @Q / y \}} \quad \text{(COMM)}$$
$$\texttt{P} \to \texttt{P'} \implies \texttt{P|Q} \to \texttt{P'|Q} \quad \text{(PAR)}$$
$$\texttt{P} = \texttt{P'}, \texttt{P'} \to \texttt{Q'}, \texttt{Q'} = \texttt{Q} \implies \texttt{P} \to \texttt{Q} \quad \text{(STRUCT)}$$

Furthermore, we can see by inspection that the model is compositional and that this composition includes an explicit representation of concurrent execution. It turns out that channels represent space usage and communication events across channels represent time usage. So this logical model has all four properties mentioned above.

4.2.1 Reasoning about dynamics

In what follows it will be convenient to write $\texttt{P} \downarrow_{\texttt{x}}$ just when $\texttt{P} = \texttt{for(y <- x)P | x!(Q) | R}$, for some R.

Likewise, it will be convenient to define the collection of contexts for the rho calculus recursively as

$\texttt{K} ::= \Box \mid \texttt{for(y <- x)K} \mid \texttt{x!(K)} \mid \texttt{P|K}$

We use contexts to characterize and label interactions. For example, when $\texttt{K} = \Box\texttt{|x!(Q)}$ we say $\texttt{for(y <- x)P}$ transitions via K to $\texttt{P\{ @Q / y \}}$, written $\texttt{for(y <- x)P} \xrightarrow{\texttt{K}} \texttt{P\{ @Q / y \}}$. Conversely, when $\texttt{K} = \texttt{for(y <- x)P|}\Box$ we have $\texttt{x!(Q)} \xrightarrow{\texttt{K}} \texttt{P\{ @Q / y \}}$.

This is merely a way of repackaging the COMM rule to get a notion of labelled transition which delivers a bisimulation which is a congruence. See [**?**] for more details.

This means we can abuse notation and write $\texttt{P} \downarrow_{\texttt{K}}$ without ambiguity. Note that when we wish to emphasize or call out the name of a context that labels a transition, we write K(x), and hence $\texttt{P} \downarrow_{\texttt{K(x)}}$.

4.3 The (spicy!) lookahead rule

As mentioned in the introduction, we modify the COMM rule as follows

$$\texttt{Q} \xrightarrow{n} \{\ \texttt{Q}_1, \ldots, \texttt{Q}_m\} \implies \texttt{for(y <- x)P | x!(Q)} \to \texttt{P\{ @\{ Q}_1, \ldots, \texttt{Q}_m\} / y \} \quad \text{(COMM)}$$

For this to make sense we need to add a minimal implementation of finite sets. That is, we need to modify the grammar to be

$$\texttt{P,Q ::= 0} \mid \texttt{x!(P)} \mid \texttt{for(y <- x)P} \mid \texttt{P|Q} \mid \texttt{*x} \mid \{\texttt{P}^*\} \qquad \texttt{x,y ::= @P}$$

with the additional equations

$$\{\ \texttt{P, Q}\ \} = \{\ \texttt{Q, P}\ \} \quad \text{and} \quad \{\ \texttt{P, P}\ \} = \{\ \texttt{P}\ \}$$

Of course, it would be considerably more useful to have a minimal sublanguage for the manipulation of sets. On the other hand, it is not strictly necessary to add sets at all, for we could simply use parallel composition to accumulate the states.

$$\texttt{Q} \xrightarrow{n} \{\ \texttt{Q}_1, \ldots, \texttt{Q}_m\} \implies \texttt{for(y <- x)P | x!(Q)} \to \texttt{P\{ @\{ Q}_1 \texttt{|} \ldots \texttt{|} \texttt{Q}_m\} / y \} \quad \text{(COMM)}$$

In rholang there are builtin primitives for set manipulation as well as pattern-matching for picking apart terms like $\texttt{Q}_1 \texttt{|} \ldots \texttt{|} \texttt{Q}_m$. Thus, while these design deliberations are of keen interest they are not the focus of the investigation here. We need a mechanism to collect the n-step fringe of evolution from Q and to deliver that collection to the continuation of the for-comprehension such that it can access the elements. In the rest of this paper we will simply assume that apparatus. Thus, for example, we will write expressions like $\texttt{for(@\{ pattern, } \ldots\ \texttt{\} <- x)P}$ and take it to extract all elements from S that match pattern when S is communicated on x, i.e. x!(S). Readers looking for more detail can see the MeTTaIL design documentation and the implementation.

4.4 What these means for rho-based agents

One of the most important developments of the mobile process calculi is to drag the environment of a computation into the computational world. We might understand this in terms of the original motivations of the π-calculus to address a significant failing of Mealy and Moore machines. In those formalisms the environment is not treated on par (pun intended) with the state machine. As a result the model is not compositional. With the parallel composition operator (`par`) we can model the environment of an agent as an agent. That is

$$\text{Agent} \mid \text{Environment}$$

Indeed, from the Environment's perspective, it is the agent and Agent is the environment. That is, if either are capable of entertaining a perspective at all.

This symmetry between environment and agent reflects something crucial about the world of computation. Computations are ontologically isolated. The only way an agent can fruitfully engage with an environment is if the events generated by the environment have a representation in the model of computation the agent is written in. Think about it this way. For a Java program to fruitfully engage the events coming from a sensor the events have to have a representation as a Java data structure. Likewise, for our brains to fruitfully engage the range of electromagnetic events that correspond to what our senses pick up, these events have to be encoded into signals the brain can process.

The whole enterprise of an agent coming to learn its environment is for it to build a model of the environment as an agent in the model of computation the agent is written in. That way it can simulate how the environment will behave, and classify its environment into various categories. What are those categories? The bisimulation equivalence classes the environment's agentic representation inhabits. These categories are faithfully described by formulae in the Hennessy-Milner logic associated with the model of computation the agent is written in.

4.4.1 The present moment

Equipped with this conceptual framework we can see that the mobile process calculi automatically deliver to an agent a present moment. How so? Let's define the surface of an agent, relative to a given environment, in terms of the following namespace

$$\mathsf{surf}(\text{agent}, \text{environment}) = \{x \in \mathsf{FN}(\text{agent}) \cap \mathsf{FN}(\text{environment}) : \text{agent} \mid \text{environment} \downarrow_x\}$$

Likewise, the agent's interior can be defined recursively. For the agent to have any internal action at all it must decompose as

$$\text{agent} = \text{agent}_1 \mid \text{agent}_2$$

such that agent_1 and agent_2 interact. We can save ourselves a lot of combinatorial futzing about related to all the ways the agent may decompose with the following definition.

$$\mathsf{int}(\text{agent}, \text{environment}) = \{x \in \mathsf{N}(\text{agent}) \setminus \mathsf{FN}(\text{environment}) : \text{agent} \downarrow_x\}$$

Now an agent's present moment comprises all the interactions it can have immediately with its environment and all the interactions it can have immediately internally with itself.

$$\mathsf{PM}(\text{agent}, \text{environment}) = \mathsf{PM}_{ext}(\text{agent}, \text{environment}) \cup \mathsf{PM}_{int}(\text{agent}, \text{environment})$$

where

$$\mathsf{PM}_{ext}(\text{agent}, \text{environment}) = \{K(x) : x \in \mathsf{surf}(\text{agent}, \text{environment}), K = \square \mid \text{environment}, \text{agent} \downarrow_{K(x)}\}$$

and

$$\mathsf{PM}_{int}(\text{agent}, \text{environment}) = \{K(x) : x \in \mathsf{int}(\text{agent}, \text{environment}), \exists \text{agent}_1, \text{agent}_2.\text{agent} = \text{agent}_1 \mid \text{agent}_2, K = \square \mid \text{agent}_2, \text{agent} \downarrow_{K(x)}\}$$

4.4.2 The future

Meanwhile an agent's future is a natural extension of its present moment to include many steps from the current state. While this is available to those agents peering from outside the calculus at the ones inside the calculus, it is not necessarily available to the agents inside the calculus unless they have a special reflective form. The `spice` rule obviates all the boilerplate associated with giving agents that form. Agents that avail themselves of $n > 0$ in their code have an interior representation of the future. Those that don't only have a *direct experience* of their present moment. To introspect on their present moment, they have to lookahead at least one step.

4.4.3 The past

The past only comes into being if there is some record or trace of the interactions an agent may have with its environment or internally. This is true of the entire calculus as well as of any particular agent. There is a transformation into a particular form for each agent, akin to continuation-passing representation in the λ-calculus, that automatically delivers this trace. The form is dubbed *continuation-saturated form* and was originally developed for the `rho calculus` to support automatic conversion of computations into reversible computations. The key idea is to intercept every action and record it and all relevant state before taking it. That's what's necessary for reversibility. It turns out to also keep a record of actions taken and any state changes that need to be remembered to put things back the way they were. In other words, agents in *continuation-saturated form* have a past.

4.4.4 Episodic memory

To see that agents are already equipped with episodic memory let's write down the generic form of an agent.

$$\text{agent} = \prod_i \text{for}(y_i \leftarrow x_i)P_i \mid \prod_j x_j!(Q_j)$$

An agent comprises a store of two kinds of things: *recipes* ($\text{for}(y_i \leftarrow x_i)P_i$) and *facts* ($x_j!(Q_j)$). Just like in human memory, and visitation of recipes and/or facts results in possible loss of the recipe and/or the fact unless it is put back. That is because visitation is interaction and interaction, by default, is linear and destructive. Therefore unless the agent has the requisite structure to restore the memory, it is used up.

5 MeTTaIL and typed communication in the rho calculus

The primary motivation of `MeTTaIL` stems from a desideratum identified by Ben Goertzel. His proposal is that AGI needs to be able to reason about different models of computation to facilitate reasoning about reasoning. At the level of programming language design we have the technology. Since Milner's seminal paper *Functions as Processes* there has been a gold standard for the presentation of operational semantics, which is codified in terms of a grammar, a collection of equations, and a collection of rewrite rules. Several formalisms, from Plotkin's Structured Operational Semantics to GSOS have been presented to capture various aspects.

`MeTTaIL` cherrypicks the best of these, notably including Fiore's Higher-Order Abstract Syntax, so that rewrite rules can be expressed over terms with binders, such as we find in the λ-calculus, the π-calculus, the `rho calculus`, etc. In modern programming languages a programmer may specify a user-defined type. In `MeTTaIL` a programmer may specify a user-defined language, where languages without rewrite rules subsumes the traditional notion of user-defined type. We call such specifications *graph-structured λ-theories* (`GSLTs` or sometimes just *theories* when the context makes it clear). This subsumes and replaces merely being able to communicate data from a data user defined data type.

Then channels can be declared to carry terms in a `GSLT`. As a consequence the `spice rule` may be used to evaluate terms in any theory. That is, if an agent written in `rho calculus` receives a term in the λ-calculus it may evaluate it using the `spice rule`, as a part of the communication. To illustrate the expressive power, let's consider simple arithmetic expressions.

```
for( y <- stdin ){ stdout!( "you typed: " + y ) } | stdin!( 1 + 1 )
```

will cause `1 + 1` to be written to the console, while

```
for( y <- stdin ){ stdout!( "you typed: " + y ) } | stdin!( 1 + 1 )[1]
```

will cause `2` to be written to the console. This makes it possible for agents to communicate models at any stage of evolution.

Let's put this into perspective. A `graph-structured` λ`-theory` is a model of causality. It says here's a proposal for how to carve up the world into states and how they evolve. In this context the `spice rule` allows agents to hypothesize about notions of causality and explore their consequences in dialogue with each other. This is necessary structure to model scientific discourse amongst a population of agents.

6 Conclusions and future work

While this approach to agentic coordination and reasoning is both compact and comprehensive in scope it is not yet as efficient to render to modern hardware as neural nets are. We are exploring different ways to render this model to vector-based computation. In particular we have a translation of `GSLTs` to *hypervectors* and hypothesize this reduces the total number of machine instructions relative to our symbolic implementation. Similarly, there is a translation of directed graphs to *upper triangular matrices* that forms the basis of a matrix interpretation of the `spice calculus`. None of this work is far enough along yet to confirm or refute whether the model enjoys efficiency gains.